

ASSYST: A Python package for automated generation of training data for machine learning interatomic potentials

Marvin Poul¹ and Jörg Neugebauer¹

¹ Max-Planck-Institute for Sustainable Materials, Germany

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#)
- [Repository](#)
- [Archive](#)

Editor: [Neea Rusch](#)

Reviewers:

- [@victorhph8](#)
- [@davidmeunier79](#)
- [@shravanngoswami](#)

Submitted: 22 May 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

The accuracy of machine-learning interatomic potentials (MLIPs) depends on high-quality training data, yet existing automation tools are often monolithic and tied to specific electronic-structure codes. ASSYST is a lightweight, pure Python library that implements the Automated Small Symmetric Structure Training method. It has a modular design: small, orthogonal classes communicate via abstract interfaces and the Atomic Simulation Environment (ASE) Calculator API. The package can be easily integrated with workflow managers (e.g. [executorlib](#) ([Janssen et al., 2025](#)) and [pyiron](#) ([Janssen et al., 2019](#))), a command-line interface (CLI) or a graphical user interface (GUI), enabling heterogeneous high-performance computing (HPC) deployment. Released under the BSD3 licence with full documentation and CI testing, ASSYST enables the flexible and extensible generation of MLIP training data across diverse research workflows.

Statement of Need

Generation of training data is the leading expense in developing new interatomic potentials, both in terms of compute time as well as human interaction. Various approaches have been developed to automate this task via active learning, though the resulting code bases are often specific to certain MLIPs and particular applications. This complicates both the comparison of methods across models and systems and the construction of foundational datasets that confer wide transferability on their MLIPs.

Recently, we developed a method, the *Automated, Small, Symmetric, Structure Training* or ASSYST. It produces training sets consisting of large numbers of small supercells, remaining agnostic to the MLIP architecture used to train on the generated data while yielding exceptionally transferable MLIPs. ([Poul et al., 2023, 2025](#)) Meanwhile, the field is rapidly advancing: universal MLIPs now model significant parts of the entire periodic table ([Lysogorskiy et al., 2026](#)), extended formalisms handle additional degrees of freedom beyond geometry ([Yang et al., 2026](#)); and new reference data expand the scope of potentials. ([Herzog et al., 2024](#); [Schran et al., 2019](#)) All of these developments hint strongly that much more and high quality training data will be required in the near future, generated and ingested by new data sources and new MLIP architectures, that are difficult to foresee. ASSYST therefore targets a design that facilitates the interoperability with different (*electronic*) reference data, MLIPs, or workflow managers.

State of the Field

There exist multiple software packages that aim to generate training data for MLIPs (and often the MLIPs themselves, too), offering integrated workflows to do so automatically or semi-automatically, e.g. ([Fletcher, n.d.](#); [Gelžinytė et al., 2023](#); [Naik et al., 2026](#); [Zeng et](#)

al., 2025) aimed at solid state materials. All of these packages are deeply integrated with the software required to run the underlying electronic structure calculations and MLIP fitting. While this can be convenient, it also makes setting up additional MLIPs or electronic data generation methods very difficult.

By comparison, *ASSYST* aims to be more compact and easier to extend, and to be less dependent on certain reference data or MLIPs. The integration of *ASSYST* has been demonstrated in multiple public workshops. (pyiron developers, 2025a, 2026a) In the supporting material of these workshopse, we demonstrated the ease of integration with existing workflow tools, MLIP codes, and a GUI—something that would be cumbersome to implement with other CLI-based tools.

Software Design and Features

This package is the result of a refactoring of a previous implementation of the method.(pyiron developers, 2025b) The previous implementation followed an integrated design pattern, similar to alternative packages mentioned above. However, on attempting to integrate that previous code with workflow tools and execution models(Huber & Neugebauer, 2026; Janssen et al., 2025; pyiron developers, 2026b) we found difficulty in cleanly expressing all options and capabilities of a monolithic code in these smaller and more flexible contexts without a lot of code duplication. Hence, we redesigned *ASSYST* from scratch by relying on:

1. small, orthogonal classes, and
2. clean, abstract interfaces between them.

The result is lightweight code that can be quickly integrated into other packages, even without in-depth knowledge of the code base. Users on High Performance Computers can benefit from this particularly because the individual steps of a workflow producing an MLIP require quite different and heterogeneous compute resources. Initial preparation tasks may require minimal parallelisation, if any, whereas electronic structure calculations are almost always heavily optimised across multiple compute nodes. Fitting the MLIP on the other hand is often best done on specialized GPU-accelerated nodes. In a loosely-coupled code like *ASSYST*, users therefore have a large flexibility to submit different parts of the workflow to the best fitting HPC resources, e.g. with (Janssen et al., 2025).

This flexibility is also crucial to future-proof the package to changes in MLIP architecture or reference data. *assyst* will require very few or no changes to deal with other MLIPs or data, while more monolithic designs, e.g. in [stateoffield] are likely to require more modifications.

In [fig1classes], we show the program design that enables the flexibility. Green boxes show individual workflow steps. Between them data flows as lists of ASE(Larsen et al., 2017) Atoms objects. Atoms are a well-known and widely used data structure in computational materials science that enables the easy distribution of the individual workflow steps. In our previous GUI implementations (pyiron developers, 2025a, 2026a), each of the boxes is represented by an interactive 'window' or 'node'. The ochre-colored boxes highlight possible and common configuration options for each step, e.g. passing `VolumeRelax("BFGS", pressure=0)` will instruct `relax(...)` to minimize the energy of each structure at zero pressure using the well-known BFGS algorithm. The violet boxes in turn show where in the workflow *ASSYST* relies on external codes to generate reference data or as a force engine. These interactions are either via the widely employed ASE Calculator interface or can be realized by overloading a minimal set of documented methods, which naturally integrate into the main workflow.

Finally, where *ASSYST* relies on random initialization or perturbation, the state of the underlying random number generators is exposed to enable fully reproducible runs. Each generated structure is assigned an ID and the relationship of derived structures is tracked. Both together enable to verify the data provenance of the each data point in a training set. This property is included in the test suite.

88 Outline of data and control flow in the package. Green boxes are the cleanly separated workflow
89 steps. Arrows among them show the flow of generated structures. Ochre boxes show example
90 code of the small modular classes that are used to configure the individual workflow steps.
91 Violet boxes show the locations where ASSYST loosely couples to external codes.

92 Research Impact Statement

93 ASSYST implements a recent and novel method to generate training data that we described
94 in detail in (Poul et al., 2023, 2025). While this implementation was not used in the original
95 publication, those papers prompted multiple independent investigations (Bienvenu et al., 2025;
96 Brunner et al., 2026; Ito et al., 2025) using derived training sets, so that we believe a unified
97 and easily extended implementation will be helpful. ASSYST has been used in multiple, public
98 teaching workshops. (pyiron developers, 2025a, 2026a)

99 Documentation and Source Code

100 Documentation include worked examples in notebooks is available at <https://assyst.readthedocs.io/stable>. The source code can be obtained under BSD 3-clause license at <https://github.com/eisenforschung/assyst>.

103 AI Usage Disclosure

104 Generative AI was used to generate the tests, parts of the documentation, and a small number of
105 features. The majority of AI contributions are clearly marked as such by the commit information
106 as authored by google-labs-jules[bot] (Google, n.d.), Claude <noreply@anthropic.com>
107 or github-bot[bot]. jules was used with Gemini 2.5 (Gemini Team, Google DeepMind, 2025)
108 and 3 (Google DeepMind, 2025), for Claude we used Sonnet 4.6 (Anthropic, 2026b) and Opus
109 4.6 (Anthropic, 2026a) and 4.7 (Anthropic, 2026c). Prompts are linked in PR descriptions where
110 applicable. Additionally, most of the test suite was generated and committed manually in early
111 commits. We ensured high quality by manual revision of all generated code and documentation.
112 These revisions were done openly on GitHub comments and can be transparently reviewed in
113 the corresponding pull request comments. The architecture of the package has been entirely
114 designed by us, and the clean and orthogonal design supports the safe use of AI because it
115 enables small localized changes that are easily reviewed by human eyes.

116 Acknowledgements

117 MP and JN acknowledge funding from the Deutsche Forschungsgemeinschaft (DFG,
118 German Research Foundation) through the Collaborative Research Center 1394 (SFB 1394,
119 No. 409476157).

120 Anthropic. (2026a). *System card: Claude opus 4.6*. Large language model system card.
121 <https://anthropic.com/claude-opus-4-6-system-card>

122 Anthropic. (2026b). *System card: Claude sonnet 4.6*. Large language model system card.
123 <https://anthropic.com/claude-sonnet-4-6-system-card>

124 Anthropic. (2026c). *System card: Claude opus 4.7*. Large language model system card.
125 <https://anthropic.com/claude-opus-4-7-system-card>

126 Bienvenu, B., Todorova, M., Neugebauer, J., Raabe, D., Mrovec, M., Lysogorskiy, Y., &
127 Drautz, R. (2025). Development of an atomic cluster expansion potential for iron and its
128 oxides. *Npj Computational Materials*, 11(1). <https://doi.org/10.1038/s41524-025-01574-w>

- 129 Brunner, F., Fioravanti, D., & Maresca, F. (2026). Liquid metal embrittlement in Fe-
130 Zn: What can we learn from atomistic simulations? *Acta Materialia*, 308, 121989.
131 <https://doi.org/10.1016/j.actamat.2026.121989>
- 132 Fletcher, V. G. (n.d.). *NS_database_builder*. [https://github.com/VGFletcher/NS_database_](https://github.com/VGFletcher/NS_database_builder)
133 [builder](https://github.com/VGFletcher/NS_database_builder)
- 134 Gelžinytė, E., Wengert, S., Stenczel, T. K., Heenen, H. H., Reuter, K., Csányi, G., &
135 Bernstein, N. (2023). Wfl python toolkit for creating machine learning interatomic
136 potentials and related atomistic simulation workflows. *The Journal of Chemical Physics*,
137 159(12). <https://doi.org/10.1063/5.0156845>
- 138 Gemini Team, Google DeepMind. (2025). *Gemini 2.5: Pushing the frontier with advanced*
139 *reasoning, multimodality, long context, and next generation agentic capabilities*. Google
140 DeepMind. [https://storage.googleapis.com/deepmind-media/gemini/gemini_v2_5_report.](https://storage.googleapis.com/deepmind-media/gemini/gemini_v2_5_report.pdf)
141 [pdf](https://storage.googleapis.com/deepmind-media/gemini/gemini_v2_5_report.pdf)
- 142 Google. (n.d.). *Jules*. <https://jules.google/>
- 143 Google DeepMind. (2025). *Gemini 3 pro model card*. Large language model card. [https://](https://deepmind.google/models/model-cards/gemini-3-pro/)
144 deepmind.google/models/model-cards/gemini-3-pro/
- 145 Herzog, B., Gallo, A., Hummel, F., Badawi, M., Bučko, T., Lebègue, S., Grüneis, A., & Rocca,
146 D. (2024). Coupled cluster finite temperature simulations of periodic materials via machine
147 learning. *Npj Computational Materials*, 10(1), 68. [https://doi.org/10.1038/s41524-024-](https://doi.org/10.1038/s41524-024-01249-y)
148 [01249-y](https://doi.org/10.1038/s41524-024-01249-y)
- 149 Huber, L., & Neugebauer, J. (2026). *Pyiron_workflow*. Zenodo. [https://doi.org/10.5281/](https://doi.org/10.5281/zenodo.19114207)
150 [zenodo.19114207](https://doi.org/10.5281/zenodo.19114207)
- 151 Ito, K., Yokoi, T., Hyodo, K., & Mori, H. (2025). Transferability of machine-learning interatomic
152 potential to α -fe nanocrystalline deformation. *International Journal of Mechanical Sciences*,
153 291–292, 110132. <https://doi.org/10.1016/j.ijmecsci.2025.110132>
- 154 Janssen, J., Surendralal, S., Lysogorskiy, Y., Todorova, M., Hickel, T., Drautz, R., &
155 Neugebauer, J. (2019). Pyiron: An integrated development environment for computational
156 materials science. *Computational Materials Science*, 163, 24–36. [https://doi.org/10.1016/](https://doi.org/10.1016/j.commatsci.2018.07.043)
157 [j.commatsci.2018.07.043](https://doi.org/10.1016/j.commatsci.2018.07.043)
- 158 Janssen, J., Taylor, M., Yang, P., Neugebauer, J., & Perez, D. (2025). Executorlib—up-scaling
159 python workflows for hierarchical heterogenous high-performance computing. *The Journal*
160 *of Open Source Software (JOSS)*, 10(108). [https://joss.theoj.org/papers/10.21105/joss.](https://joss.theoj.org/papers/10.21105/joss.07782)
161 [07782](https://joss.theoj.org/papers/10.21105/joss.07782)
- 162 Larsen, A. H., Mortensen, J. J., Blomqvist, J., Castelli, I. E., Christensen, R., Duřak, M.,
163 Friis, J., Groves, M. N., Hammer, B., Hargus, C., Hermes, E. D., Jennings, P. C.,
164 Jensen, P. B., Kermode, J., Kitchin, J. R., Kolsbjerg, E. L., Kubal, J., Kaasbjerg, K.,
165 Lysgaard, S., ... Jacobsen, K. W. (2017). The atomic simulation environment—a python
166 library for working with atoms. *Journal of Physics: Condensed Matter*, 29(27), 273002.
167 <http://stacks.iop.org/0953-8984/29/i=27/a=273002>
- 168 Lysogorskiy, Y., Bochkarev, A., & Drautz, R. (2026). Graph atomic cluster expansion
169 for foundational machine learning interatomic potentials. *Npj Computational Materials*.
170 <https://doi.org/10.1038/s41524-026-01979-1>
- 171 Naik, A. A., Ertural, C., George, J., Liu, Y., Fragapane, N., Morrow, J., Sobolev, A.,
172 Deringer, V., Zhou, Y., & Sharan, M. (2026). *Autoatml/autoplex: v0.3.1*. Zenodo.
173 <https://doi.org/10.5281/zenodo.14105049>
- 174 Poul, M., Huber, L., Bitzek, E., & Neugebauer, J. (2023). Systematic atomic structure
175 datasets for machine learning potentials: Application to defects in magnesium. *Physical*
176 *Review B*, 107(10). <https://doi.org/10.1103/PhysRevB.107.104103>

- 177 Poul, M., Huber, L., & Neugebauer, J. (2025). Automated generation of structure datasets
178 for machine learning potentials and alloys. *Npj Computational Materials*, 11(1). <https://doi.org/10.1038/s41524-025-01669-4>
179
- 180 pyiron developers. (2025a). *ASSYST tutorial, DPG spring meeting 2025*. https://workshop.pyiron.org/DPG-tutorial-2025/02_assyst.html
181
- 182 pyiron developers. (2025b). *Pyiron_potentialfit*. https://github.com/pyiron/pyiron_potentialfit
183
- 184 pyiron developers. (2026a). *ASSYST tutorial, DPG spring meeting 2026*. https://workshop.pyiron.org/DPG-tutorial-2026/03_assyst.html
185
- 186 pyiron developers. (2026b). *Pyiron_core*. https://github.com/pyiron/pyiron_core
- 187 Schran, C., Behler, J., & Marx, D. (2019). Automated fitting of neural network potentials at
188 coupled cluster accuracy: Protonated water clusters as testing ground. *Journal of Chemical*
189 *Theory and Computation*, 16(1), 88–99. <https://doi.org/10.1021/acs.jctc.9b00805>
- 190 Yang, J., Li, B., Mattoso, S., Abdelkawy, A., Todorova, M., & Neugebauer, J. (2026).
191 Macro-dipole-constrained learning of atomic charges for accurate electrostatic potentials at
192 electrochemical interfaces. *Physical Review Letters*, 136(11), 118001. <https://doi.org/10.1103/5hh2-wypm>
193
- 194 Zeng, J., Zhang, D., Peng, A., Zhang, X., He, S., Wang, Y., Liu, X., Bi, H., Li, Y.,
195 Cai, C., Zhang, C., Du, Y., Zhu, J.-X., Mo, P., Huang, Z., Zeng, Q., Shi, S., Qin,
196 X., Yu, Z., ... Wang, H. (2025). DeePMD-kit v3: A Multiple-Backend Framework for
197 Machine Learning Potentials. *J. Chem. Theory Comput.*, 21(9), 4375–4385. <https://doi.org/10.1021/acs.jctc.5c00340>
198